

Department of Computer Science
University of Pretoria

Programming Languages
COS 333

Practical 2: Functional Programming

23 February, 2026

1 Objectives

This practical aims to achieve the following general learning objectives:

- To gain and consolidate some experience writing functional programs in Scheme;
- To consolidate the concepts covered in Chapter 15 of the prescribed textbook.

2 Plagiarism Policy

Plagiarism is a serious form of academic misconduct. It involves both appropriating someone else's work and passing it off as one's own work afterwards. Thus, you commit plagiarism when you present someone else's written or creative work (words, images, ideas, opinions, discoveries, artwork, music, recordings, computer-generated work, etc.) as your own. Note that using material produced in whole or part by an AI-based tool (such as ChatGPT) also constitutes plagiarism. Only hand in your own original work. Indicate precisely and accurately when you have used information provided by someone else. Referencing must be done in accordance with a recognized system. Indicate whether you have downloaded information from the Internet. For more details, visit the library's website: <http://www.library.up.ac.za/plagiarism/>.

3 Submission Instructions

Upload your practical-related source code file to the appropriate assignment upload slot on the ClickUP course page. You will be implementing all the practical's tasks in the same file. Name this file `u99999999.scm`, where `99999999` is your student number. Multiple uploads are allowed, but only the last one will be marked. The submission deadline is **Monday, 9 March 2026, at 12:00**.

4 Background Information

For this practical, you will be writing functional programs in Scheme:

- You will have to use the DrRacket 8.9 IDE, which is installed on the Windows computers in the Informatorium. You should write your function implementations in DrRacket. Note that your program must always start with the line:

```
#lang racket
```

You can then run the source file by clicking on the “Run” icon at the top of the screen. You do not need to include the code to test your functions at the end of your program, but the tutors will require you to enter test code during your demo.

- A reference manual for MIT/GNU Scheme [1] has been uploaded on the course website. Please also refer to chapter 15 of the textbook, and the slides that have been uploaded for that chapter. Also note that the sisp implementation of Scheme uses lowercase letters for built-in function names (e.g. `odd?` rather than `ODD?`), and doesn’t provide all the functions discussed in the slides and textbook (e.g. `<>` is not provided).
- **Note that you may only use the following built-in Scheme functions and language features in your programs. Racket provides additional functionality that Scheme does not, and you may also not use any of these language features. Failure to observe these rule will result in all marks for a task being forfeited:**

Function construction: `lambda`, `define`

Binding: `let` (more complex `let` functions, liked `let*` and a named `let` are not allowed)

Arithmetic: `+`, `-`, `*`, `/`, `abs`, `sqrt`, `remainder`, `modulo`, `min`, `max`

Boolean values: `#t`, `#f`

Equality predicates: `=`, `>`, `<`, `>=`, `<=`, `even?`, `odd?`, `zero?`, `positive?`, `negative?`, `eqv?`, `eq?`

Logical predicates: `and`, `or`, `not`

List predicates: `list?`, `null?`

Conditionals: `if`, `cond`, `else`

Quoting: `quote`, `'`

Evaluation: `eval`

List manipulation: `list`, `car`, `cdr`, `cons`

Input and output: `display`, `printf`, `newline`, `read`

5 Practical Tasks

For this practical, you will need to explore and implement functional programming concepts, including list processing and the use of recursive functions. This practical consists of three tasks, and all three tasks should be implemented in a single source code file.

5.1 Task 1

Write a function named `coneVolume`, which receives two parameters. The first parameter is a numeric atom representing the radius of the circular base of the cone. The second parameter is a numeric atom representing the height of the cone. The function should yield (not print out) the volume of the cone with the provided radius and height, computed as:

$$V = \pi r^2 \frac{h}{3}$$

where r is the radius of the cone’s base and h is the height of the cone. The function should yield zero if either r or h is zero or negative. Use a `let` (not a `define`) to bind the name `pi` to the result of 22 divided by 7. Note that `22/7` is a fraction literal in Scheme, not a division. To achieve a division, use the arithmetic division function covered in the lectures on Chapter 15. For example, the function application

```
(coneVolume 1.5 2.1)
```

should yield a result of 4.95. **Be sure to thoroughly test your function.** Do not use only the provided function applications. Test all the possible edge cases you can think of. **Only use the prescribed language features and functions provided in Section 4.**

5.2 Task 2

Write a function named `countNonZeroValues`, which receives one parameter. The parameter is a simple list (i.e. a list containing only atoms). You may assume that the list contains only integer valued numeric atoms. This list should then be searched for non-zero values. The `countNonZeroValues` function should yield (not print out) the total number of these non-zero values. For example, the function application

```
(countNonZeroValues '())
```

should yield 0, because the second parameter contains no values (including non-zero values). As another example, the function application

```
(countNonZeroValues '(0 0))
```

should also yield 0, because the list contains only zero values (and therefore no non-zero values). As a final example, the function application

```
(countNonZeroValues '(3 0 -8 0))
```

should yield 2 because 3 and -8 are the only non-zero values contained in the list, and this constitutes two separate values.

To implement the `countNonZeroValues` function, you will have to recursively traverse the parameter list, computing a count as you do so. You may make use of a helper function if you choose to. However, note that it is possible to implement `countNonZeroValues` as a single function, and try to do so as an exercise. **Be sure to thoroughly test your function.** Do not use only the provided function applications. Test all the possible edge cases you can think of. **Only use the prescribed language features and functions provided in Section 4.**

5.3 Task 3

Write a function named `doubleEverySecondElement`, which receives one parameter. The parameter is a simple list (i.e. a list containing only atoms). You may assume that the list contains only integer valued numeric atoms. The function should yield a list containing the double of only every second value in the parameter list. For example, the function application

```
(doubleEverySecondElement '())
```

should yield an empty list, because the parameter list contains no values. As another example, the function application

```
(doubleEverySecondElement '(4 6))
```

should also yield the list (12), because the second element in the list is the value 6, and 6 doubled is 12. As a final example, the function application

```
(doubleEverySecondElement '(4 6 2 3))
```

should yield the list (12 6) because the second element and fourth elements are 6 and 3, and these values doubled are 12 and 6, respectively.

To implement the `doubleEverySecondElement` function, you will have to recursively traverse the parameter list, and build up a result list. **Be sure to thoroughly test your function.** Do not use only the provided function applications. Test all the possible edge cases you can think of. **Only use the prescribed language features and functions provided in Section 4.**

Hint: Consider writing multiple functions for this task, where each function performs a different part of the required processing on the parameter list.

6 Marking

Each of the tasks will count 5 marks for a total of 15 marks. Submit all three tasks implemented in the same source code file. Do not upload any additional files other than your source code. Both the implementation and the correct execution of the functions will be taken into account. **You will receive zero for a task that uses a language feature you are not allowed to use.** Your program code will be assessed during the practical sessions in the week of **Monday, 9 March 2026.**

References

- [1] Chris Hanson and the MIT Scheme Team. MIT/GNU Scheme reference manual. Technical report, Massachusetts Institute of Technology, 2018.